ZEROTRACE: UNIFIED SYSTEM ARCHITECTURE & FILE HIERARCHY

Version 2.0 (Updated) | Author: Aniruddh Gupta | Target: Single Laptop (16GB RAM) | Year: 2026

"They leave no trace. Because we found them first."

1. Executive Summary & Philosophy

ZeroTrace is an open-source, zero-cost, CPU-optimized cyber defense platform combining three modular traces into a single source of truth:

- **Trace 1 (RECON)**: Attack Surface Discovery (Subdomains, Ports, S3 buckets, SSL expiry, Shadow IT).
- **Trace 2 (INTEL)**: AI-Powered Vulnerability Prioritization (NVD + EPSS, XGBoost Exploit Model, Ollama Auto-Fix).
- **Trace 3 (GUARD)**: LLM Security Middleware Gateway (Regex pre-filter, ONNX DistilBERT classifier, PII scanner, Hash-chained SQLite audit log).

Operational Readiness Score (v2.0 Fixed) = 100 - (0.40 × Asset Exposure + 0.40 × Vuln Urgency + 0.20 × AI Threat Exposure Index)

2. Complete File Hierarchy v2.0

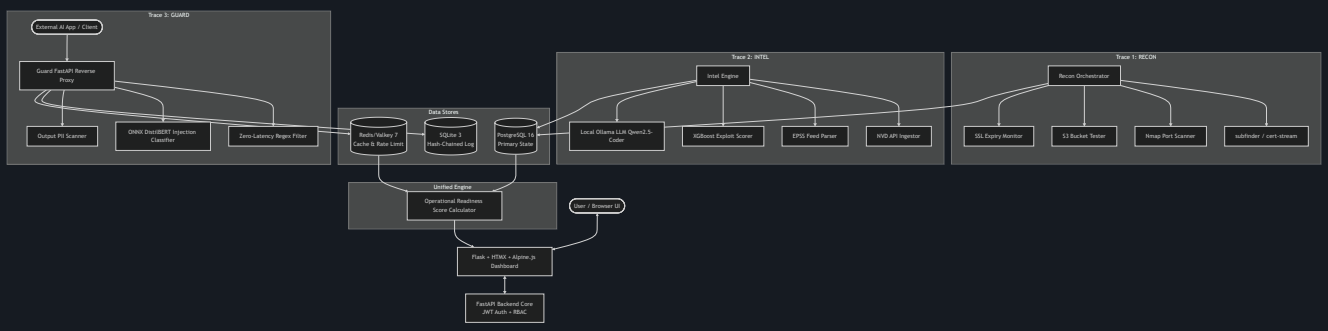
```
zerotrace/
├── docker-compose.yml              # Multi-container orchestration (DB, Redis, API, Da
├── Dockerfile.api                  # Docker build file for FastAPI backend engine
├── Dockerfile.dashboard            # Docker build file for Flask/HTMX dashboard
├── .env.example                    # Environment variable templates (DB credentials, A
├── .gitignore                      # Exclude bytecode, ONNX weights, local SQLite DB,
```

```
├── Makefile                        # Dev commands (make run, make test, make db-migrat
├── requirements.txt                # Python dependencies (FastAPI, Flask, SQLAlchemy,
├── README.md                       # Comprehensive project setup and execution manual
│
├── config/                         # Configuration Management System
│   ├── __init__.py
│   ├── settings.py                 # Pydantic BaseSettings loading .env config
│   ├── logging_config.py           # Structured JSON logging configuration
│   └── database.py                 # SQLAlchemy async engine, session maker, Redis cli
│
├── db/                             # Relational Data Layer (PostgreSQL & Alembic)
│   ├── alembic.ini                 # Database migration configuration
│   ├── migrations/                 # Alembic migration scripts directory
│   │   ├── env.py
│   │   └── versions/
│   │       └── 0001_initial_schema.py
│   └── models/                     # SQLAlchemy ORM Data Models
│       ├── __init__.py
│       ├── asset.py                # Asset table (IP, Hostname, Ports, Services, S3, S
│       ├── vulnerability.py        # CVE, CVSS, EPSS score, ML Exploit Score, Patch St
│       ├── llm_audit.py            # Guard proxy request records (Prompt hash, score,
│       ├── readiness.py            # Historical Operational Readiness Score snapshots
│       └── user.py                 # User authentication model (Username, bcrypt passw
│
├── auth/                           # Authentication & Authorization Subsystem [NEW v2.
│   ├── __init__.py
│   ├── jwt_handler.py              # JWT token creation, decoding, validation, and ref
│   ├── session_manager.py          # Flask-Login session authentication manager
│   ├── rbac.py                     # Role-Based Access Control decorator (Viewer, Oper
│   └── password_utils.py           # Bcrypt hashing & verification utilities
│
├── modules/                        # Core Domain Logic (Three Traces)
│   ├── __init__.py
│   │
│   ├── recon/                      # TRACE 1: RECON MODULE (Attack Surface Discovery)
│   │   ├── __init__.py
│   │   ├── subdomains.py           # Subdomain enumeration engine (subfinder + cert-st
│   │   ├── port_scanner.py         # Nmap top-100 port & service banner discovery wrap
│   │   ├── s3_checker.py           # AWS S3 public bucket misconfiguration tester
│   │   ├── ssl_monitor.py          # SSL/TLS certificate expiration & protocol checker
│   │   ├── shadow_it.py            # Inventory lookup & unauthorized asset flagger
│   │   └── runner.py               # Orchestrator running full recon workflow async wi
│   │
│   ├── intel/                      # TRACE 2: INTEL MODULE (Vulnerability Prioritizati
│   │   ├── __init__.py
│   │   ├── nvd_feed.py             # NVD API ingestor for CVE vulnerability data (with
│   │   ├── epss_feed.py            # EPSS (Exploit Prediction Scoring System) daily CS
```

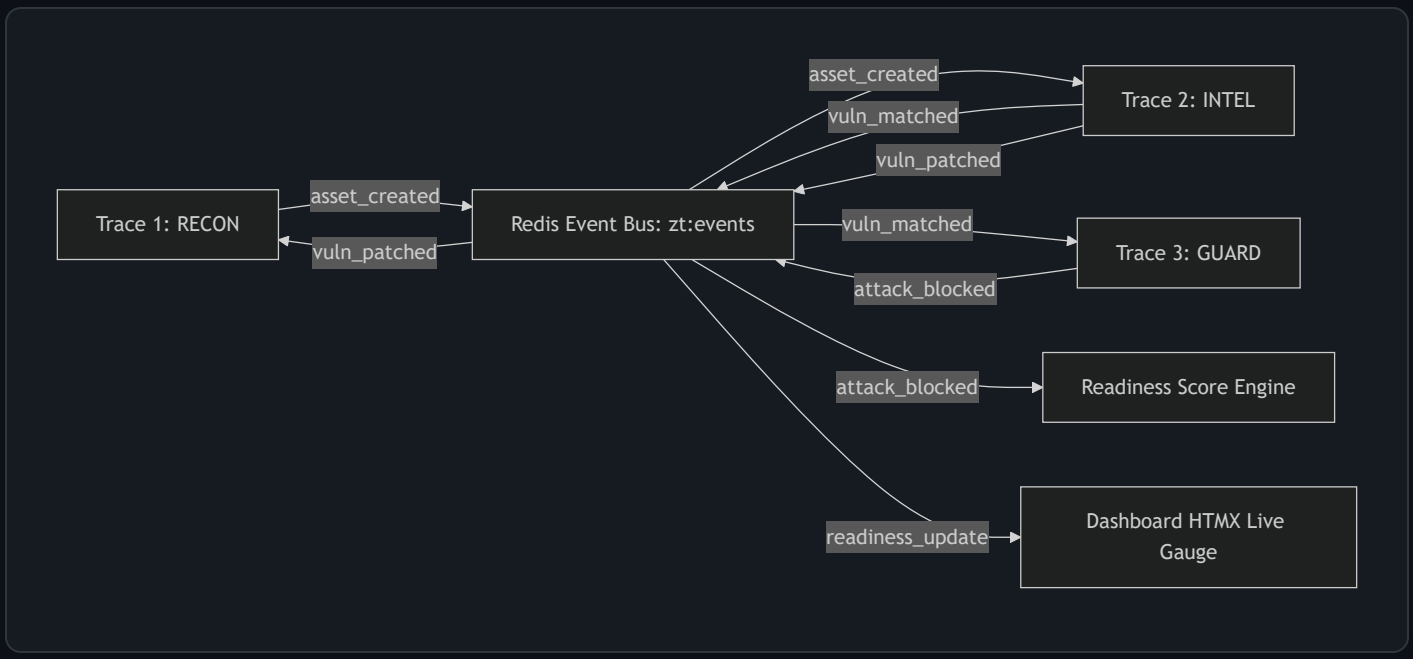
```
│   │   ├── cpe_matcher.py          # Service banner to CPE 2.3 string & CVE mapper
│   │   ├── ml_scorer.py            # XGBoost ML inference engine for exploit probabili
│   │   ├── patch_queue.py          # 48-hour prioritized remediation queue generator
│   │   └── autofix_llm.py          # Local Ollama Qwen2.5-Coder prompt builder & code
│   │
│   └── guard/                      # TRACE 3: GUARD MODULE (LLM Security Middleware)
│       ├── __init__.py
│       ├── proxy.py                # FastAPI reverse proxy endpoint intercepting /v1/c
│       ├── regex_filter.py         # Fast zero-latency heuristic & regex rule checker
│       ├── injection_classifier.py  # ONNX Runtime DistilBERT semantic prompt injectio
│       ├── pii_scanner.py          # Outbound response scanner for API keys, SSNs, and
│       ├── sqlite_logger.py        # Append-only SHA-256 hash-chained tamper-resistant
│       └── rate_limiter.py         # Redis token-bucket rate limiter per client ID
│
├── core/                           # Unified System Core Engine
│   ├── __init__.py
│   ├── readiness_calculator.py     # Directionally-consistent Operational Readiness Sc
│   ├── event_bus.py                # Redis Pub/Sub async event dispatcher (zt:events)
│   └── redis_cache.py              # High-frequency query caching helper
│
├── api/                            # Backend API Service (FastAPI)
│   ├── __init__.py
│   ├── main.py                     # FastAPI entrypoint, CORS, security headers, JWT m
│   ├── routers/
│   │   ├── recon_router.py         # Endpoints for triggering & querying Recon scans
│   │   ├── intel_router.py         # Endpoints for CVE query, patch queue, auto-fix ge
│   │   ├── guard_router.py         # Interception proxy + Guard log query endpoints
│   │   ├── readiness_router.py     # Operational Readiness Score & metrics API
│   │   └── auth_router.py          # Login, logout, JWT refresh, user management endpo
│   └── schemas/                    # Pydantic Request/Response Models
│       ├── recon_schema.py
│       ├── intel_schema.py
│       ├── guard_schema.py
│       ├── readiness_schema.py
│       └── auth_schema.py          # LoginRequest, TokenResponse, UserCreate schemas |
│
├── dashboard/                      # Front-end UI (Flask + HTMX + Alpine.js)
│   ├── app.py                      # Flask app handling session auth & rendering HTMX
│   ├── static/
│   │   ├── css/main.css            # Dark-mode cybersecurity UI styling (glassmorphism
│   │   ├── js/app.js              # Alpine.js state store & HTMX event handlers
│   │   └── vendor/                 # Offline vendor packages (No Node.js / npm needed)
│   │       ├── htmx.min.js         # HTMX 1.9.x (14KB)
│   │       └── alpine.min.js       # Alpine.js 3.x (15KB)
│   └── templates/
│       ├── base.html               # Main HTML layout, head, scripts, navbar
│       ├── login.html              # CSRF-protected login page [NEW v2.0]
```

```
│       ├── index.html              # Command Center Unified View
│       └── components/
│           ├── readiness_gauge.html # Animated SVG score ring (HTMX auto-refresh via e
│           ├── recon_panel.html    # Discovered assets & open ports table
│           ├── intel_panel.html    # Prioritized patch queue & auto-fix modal trigger
│           ├── guard_panel.html    # Real-time LLM attack logs & block counters
│           └── autofix_modal.html  # Syntax-validated script display
│
├── models_weights/                 # Model Artifacts (Downloaded at setup, NOT in Git)
│   ├── distilbert_injection.onnx   # Quantized DistilBERT model (~65MB)
│   ├── tokenizer_config.json       # HuggingFace tokenizer configs for DistilBERT
│   └── xgboost_exploit_model.pkl   # Pre-trained XGBoost exploit classifier weights
│
├── scripts/                        # Utility & Setup Automation Scripts
│   ├── download_models.py          # Script to fetch ONNX model & tokenizer files
│   ├── seed_database.py            # Populates DB with initial sample asset & threat d
│   ├── create_admin.py             # Script to create initial admin user [NEW v2.0]
│   └── start_services.sh           # Docker entrypoint starter script
│
└── tests/                          # Automated Test Suite (pytest)
    ├── conftest.py                 # Test fixtures & database mock setup
    ├── test_recon.py               # Unit tests for subdomain, nmap, S3, SSL scanners
    ├── test_intel.py               # Unit tests for CPE matching, XGBoost scoring, Oll
    ├── test_guard.py               # Unit tests for Regex filter, ONNX classifier, PII
    ├── test_readiness.py           # Unit tests for Readiness score formula & monotoni
    ├── test_auth.py                # Tests for JWT creation, password hashing, RBAC [N
    └── test_integration.py         # End-to-end integration tests [NEW v2.0]
```

3. Master Architecture Topology



4. Cross-Module Event Bus Flow (Redis Pub/Sub)



5. Memory Footprint Allocation

Component / Service	RAM Usage	Status / Optimization
Windows 11 OS	~4.0 GB	Background OS
WSL 2 (Ubuntu + Docker)	~2.0 GB	Linux Subsystem
PostgreSQL 16	~500 MB	Primary Relational DB
Redis (Valkey 7)	~300 MB	Cache, Rate Limiter, Event Bus
FastAPI Backend	~600 MB	Async API Engine
Ollama (Qwen2.5-Coder 7B Q4_K_M)	~4.5 GB	Quantized Local LLM
ONNX Runtime (DistilBERT)	~400 MB	CPU Inference Engine
Flask Dashboard	~200 MB	Presentation Tier

Component / Service	RAM Usage	Status / Optimization
Total Active Memory	**~9.0 GB**	**Comfortable on 16GB–20GB RAM**
